

COMP 1324 Algorithmics

Tutorial 7

Hikmat Farhat

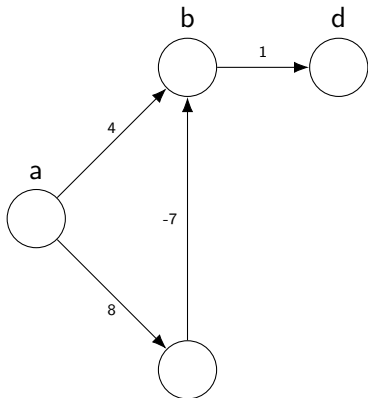
- ① An undirected graph $G = (V, E, w)$ contains some edges with negative weight but no negative weight cycles.
 - ① Can we still use Kruskal's algorithm to find a MST? What changes are needed?
 - ② Can we still use Dijkstra's algorithm to find the shortest path between two nodes? What changes are needed?
- ② Propose an algorithm to find the longest path in a directed acyclic graph.
- ③ What is the complexity of DFS if the graph is represented by an adjacency matrix?

Question 1 Solution: negative weight edges

- For Kruskal's algorithm, yes. Let c be the magnitude of the smallest weight edge. Add c to all edges. This makes all the edge weights positive and, crucially, have the same relative order. The resulting tree is the MST of the original graph.
- For Dijkstra's algorithm, no. The reason is that the shortest path between two nodes may not be the path with the fewest edges.

Question 1 Solution: negative weight edges

- Does not work Dijkstra's because different paths can have a different number of edges and therefore we don't add the same constant to all the the paths. Adding 8 to all edges changes the cost of the original shortest path from a to d ($a \rightarrow c \rightarrow b \rightarrow d$) from 2 to 26 and it is no longer the shortest path.



Question 2 Solution: Longest path in a DAG

- Replace the weights by their negative values and run Bellman-Ford's algorithm.
- A DAG has no cycles and therefore the algorithm will terminate.
- The shortest path in the "negative graph" is the longest path in the original graph.
- This works also as long as the original graph does not contain a "positive" cycle.
- The general case is NP-hard.

Question 3 Solution: complexity of DFS with adjacency matrix

- As before DFS-VISIT is called exactly once for each vertex.
- But with an adjacency matrix the cost of DFS-VISIT for vertex u is $\Theta(V)$ instead of $|adj[u]|$.
- Therefore the total complexity is $\Theta(V^2)$ (initialisation costs $\Theta(V)$).

```
1 function DFS-VISIT(adj, u)
2     .....
3     foreach  $v \in adj[u]$  do
4         if  $v.color = WHITE$  then
5              $v.p \leftarrow u$ 
6             DFS-VISIT(adj, v)
7     .....
```

- 4 Propose a modification to the DFS algorithm to detect cycles in a graph.
- 5 Given a 2-d array, `int[][] grid`, whose values are either 1 (land) or 0 (water), write a function to return the number of islands in the grid. An island is a land surrounded by water and is formed by connecting adjacent lands horizontally or vertically (not diagonally). Assume that the grid itself is surrounded by water.

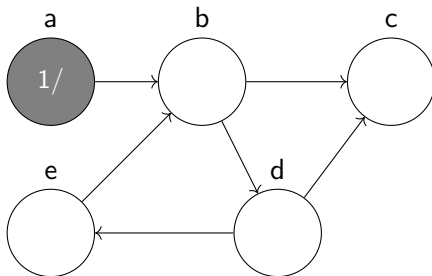
```
1  int [][] grid={
2      {1,1,0,0,0},
3      {1,1,0,0,0},
4      {0,0,1,0,0},
5      {0,0,0,1,1},
6      };
7  numIslands(grid); //should return 3
```

Q4 Solution: Detecting Cycles in graphs

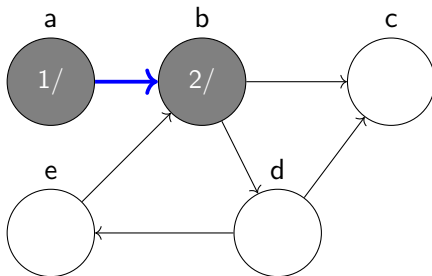
- A graph contains a cycle iff it contains a back edge
- An edge (u, v) is said to be a back edge if v is an ancestor of u .
- v is an ancestor of u iff v has a gray color.
- Why not black also? See next slides for a counter example.

```
1 function DFS-VISIT(adj, u)
2   .....
3   foreach  $v \in \text{adj}[u]$  do
4     if  $v.\text{color} = \text{WHITE}$  then
5        $v.p \leftarrow u$ 
6       DFS-VISIT(adj,  $v$ )
7     else if  $v.\text{color} = \text{GRAY} \wedge v \neq u.p$  then
8        $\text{cycle} \leftarrow \text{true}$ 
9     .....
```

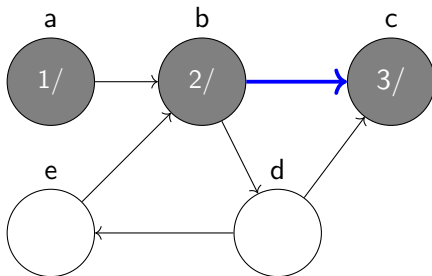
Why not black?



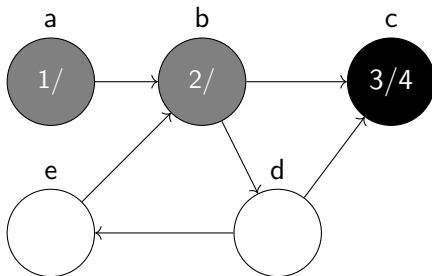
Why not black?



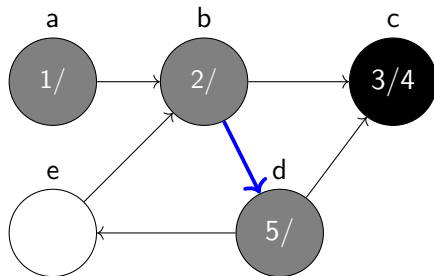
Why not black?



Why not black?

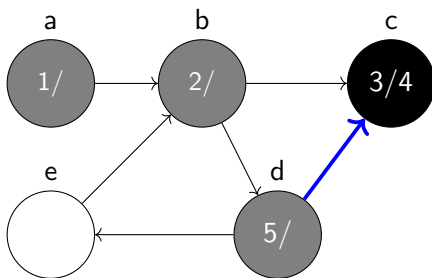


Why not black?

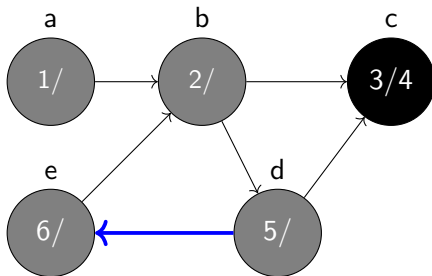


Why not black?

No cycle

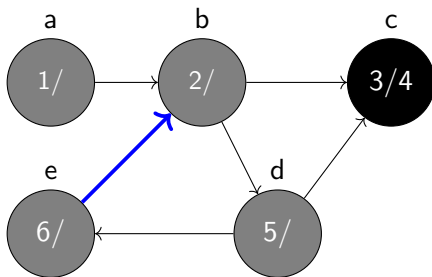


Why not black?

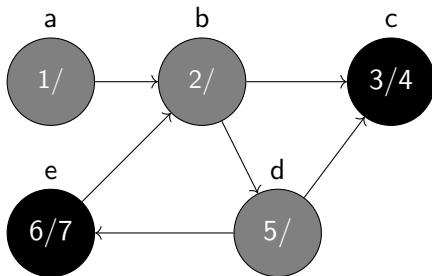


Why not black?

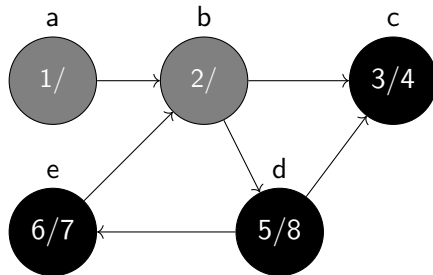
Cycle



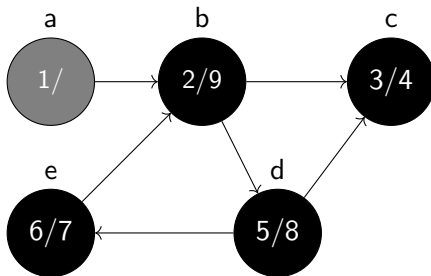
Why not black?



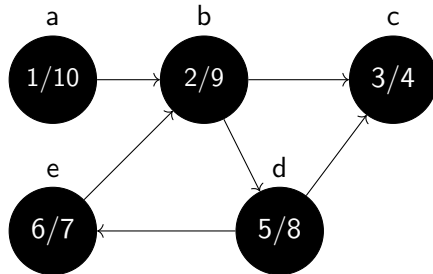
Why not black?



Why not black?



Why not black?



Question 5 Solution: Number of Islands

- This is basically determining the number of connected components in an **undirected** graph and we can use a DFS to visit all the nodes in a connected component.
- Instead of building adjacency lists, we can use the grid itself to represent the graph.
- In this case the adjacencies of a node (i, j) are the nodes $(i - 1, j)$, $(i + 1, j)$, $(i, j - 1)$, $(i, j + 1)$.
- Respect boundary conditions, i.e., do not visit nodes outside the grid.
- Set the value of a visited node to 0.

Q5 Solution: Number of Islands

```
1 int numIslands(int [][] grid ,int n,int m) {
2     int count = 0;
3     for (int i = 0; i < n; ++i) {
4         for (int j = 0; j < m; ++j) {
5             if (grid[i][j] == '1') {
6                 count++;
7                 visit(grid , i , j ,n,m);
8             }
9         }
10    }
11    return count;
12 }
```

Q5 Solution: Number of Islands

Visit all the adjacent nodes of (i,j) if not already marked and mark them as visited.

```
1 void visit(int [][] grid, int i,
2           int j, int n, int m) {
3     if (grid[i][j] == '0') return;
4     grid[i][j] = '0';
5     if (i != 0 && (grid[i-1][j] == '1'))
6         visit(grid, i-1, j);
7     if (j != 0 && (grid[i][j-1] == '1'))
8         visit(grid, i, j-1);
9     if (i != n-1 && (grid[i+1][j] == '1'))
10        visit(grid, i+1, j);
11    if (j != m-1 && (grid[i][j+1] == '1'))
12        visit(grid, i, j+1);
13
14 }
```

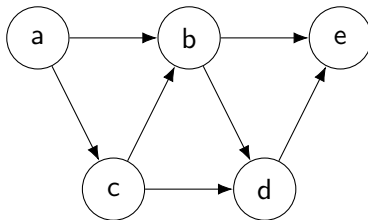
- 6 Given a directed graph $G = \langle V, E \rangle$, a source vertex $s \in V$, destination vertex $d \in V$, write an algorithm to find the number of paths from s to d having exactly m edges (Hint: try BFS without marking visited nodes).

Example, in the graph shown below there are 3 paths from a to e with length 3:

$a \rightarrow b \rightarrow d \rightarrow e$

$a \rightarrow c \rightarrow b \rightarrow e$

$a \rightarrow c \rightarrow d \rightarrow e$



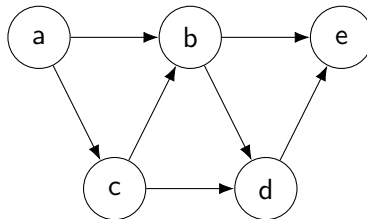
Question 6 Solution: Number of paths of length m

- Allow BFS to visit a node more than once, every time creating a copy of the node.

```
1 function NumPaths( $G, s, dest, m$ )
2   ENQUEUE( $Q, s$ );
3   while  $Q \neq \emptyset$  do
4      $u \leftarrow$  DEQUEUE( $Q$ );
5     if  $u.d > m$  then
6       break;
7     if  $u.label = dest.label \wedge u.d = m$  then
8        $count++$ ;
9     foreach  $v \in adj[u]$  do
10       $z \leftarrow copy(v)$ ;
11       $z.d \leftarrow u.d + 1$ ;
12      ENQUEUE( $Q, z$ );
```

Question 3 Solution: Number of paths of length m

$a(0)$

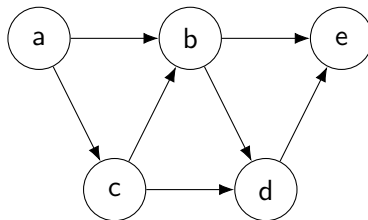


Question 3 Solution: Number of paths of length m

$b(1)c(1)$

count=0

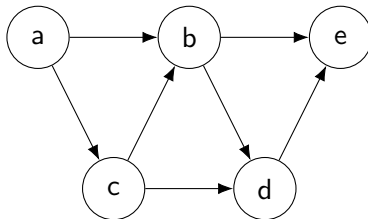
processing $\rightarrow a(0)$



Question 3 Solution: Number of paths of length m

c(1)d(2)e(2) count=0

processing → b(1)

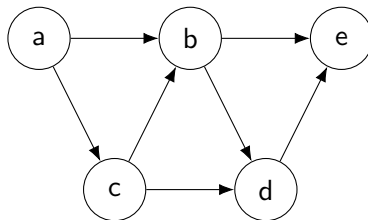


Question 3 Solution: Number of paths of length m

d(2)e(2)b(2)d(2)

count=0

processing $\rightarrow$ c(1)

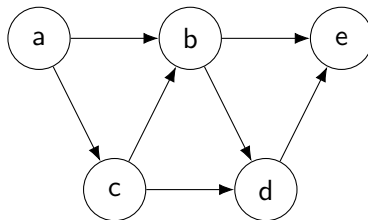


Question 3 Solution: Number of paths of length m

e(2)b(2)d(2)e(3)

count=0

processing $\rightarrow$ d(2)

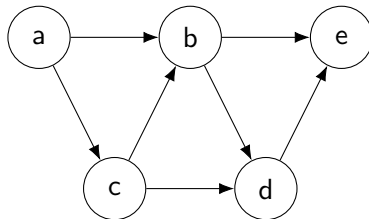


Question 3 Solution: Number of paths of length m

b(2)d(2)e(3)

count=0

processing $\rightarrow$ e(2)

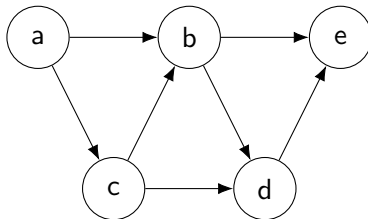


Question 3 Solution: Number of paths of length m

d(2)e(3)d(3)e(3)

count=0

processing $\rightarrow$ b(2)

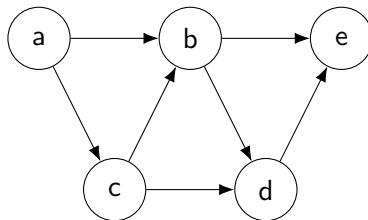


Question 3 Solution: Number of paths of length m

e(3)d(3)e(3)e(3)

count=0

processing $\rightarrow$ d(2)

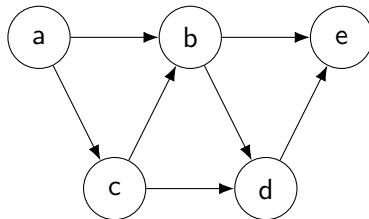


Question 3 Solution: Number of paths of length m

d(3)e(3)e(3)

count=1

processing $\rightarrow$ e(3)

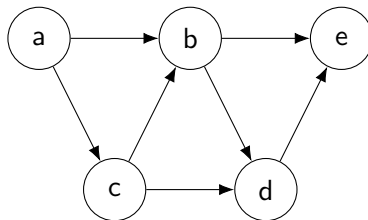


Question 3 Solution: Number of paths of length m

$e(3)e(3)$

count=1

processing $\rightarrow d(3)$

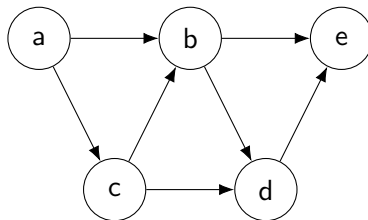


Question 3 Solution: Number of paths of length m

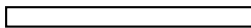
$e(3)$

count=2

processing $\rightarrow e(3)$



Question 3 Solution: Number of paths of length m



count=3

processing $\rightarrow e(3)$

